PDF Study Buddy: Project Documentation

1. Goal

Build a chatbot that answers from one PDF only, makes quizzes and summaries, and picks the right action by itself. It must be honest: when the PDF does not contain the answer, it says so.

The project combines three topics:

- **RAG:** find the right pieces of the PDF, then answer from them.
- **Gen AI:** a local language model writes answers, quizzes and summaries.
- **Agentic AI:** a router chooses a tool for each message.

2. Architecture

```
            ┌──────────────────── building the knowledge base (once per PDF) ────────────────────┐
PDF ─► read + clean ─► cut references ─► chunks ─► embeddings ─► Pinecone

            ┌──────────────────── answering a question
──────────────────────────────────────────────┐
question ─► embedding ─► Pinecone (12 close chunks) ─► reranker (best 4)
         ─► guard ("I don't know" if the best score is low)
         ─► LLM answers using only those chunks
         ─► grounding check (replace the answer with a quote if it uses outside knowledge)

            ┌──────────────────── the agent
──────────────────────────────────────────────────────────────┐
message ─► router (LLM picks a tool, keyword rules as backup)
        ─► answer | quiz | summary | show answers
```

3. The notebook, part by part

Part 1: Setup. Installs the libraries, starts Ollama and downloads the model, loads the embedding model, connects to Pinecone. The model name lives in one variable (`MODEL`).

Part 2: Load and clean the PDF. Reads each page and keeps its number. Removes repeated headers and footers, fixes joined letters (fi → fi), removes citation marks like `[12]`, and joins words split across lines. Cuts everything from the "References" heading onwards, because a list of paper titles confuses search. Marks section headings (short lines like "Chunking") so each section starts a new chunk.

Part 3: Chunking. The model cannot read a whole PDF for every question, so the text is cut into pieces of about 150 words. Whole sentences are packed together (a chunk never stops mid-sentence), consecutive chunks share about 30 words, and chunks never cross a page, so each keeps an exact page number.

Part 4: Vector database. Each chunk becomes 384 numbers (an embedding) that capture its meaning. Pinecone stores the numbers with the page and the text. Old data is deleted first, so two PDFs never mix.

Part 5: Search with reranking. Pinecone quickly returns 12 chunks that are close in meaning. A reranker then reads the question and each chunk together and re-orders them. The best 4 are kept. Without the reranker, a chunk that only mentions a topic could outrank the chunk that explains it.

Part 6: The answer function. The core RAG loop:

1. Retrieve the best chunks.
2. Guard: if even the best chunk is a poor match, answer "I don't know" without calling the model.
3. Give the model only the relevant chunks, and tell it to answer only from them.
4. Grounding check: measure how many of the answer's key words appear in the chunks. If too few do, the model probably used its own knowledge. The bot then asks it to copy the supporting sentences, checks they really exist in the PDF, and shows them as "Closest text in the PDF" (or says "I don't know").

Part 7: Evaluation. 46 questions: 38 the PDF answers (with the correct page and keywords) and 8 it should refuse (6 off-topic, 2 traps that sound on-topic). It measures retrieval (is the right page in the top 4?), answers (right keywords, not "I don't know"), and refusals. The "I don't know" cutoff is set from these scores instead of guessed. Evaluation runs at temperature 0 so repeated runs give the same result.

Part 8: Quiz tool. The model writes one question per passage with an evidence sentence copied from the PDF. A question must pass these checks before the user sees it:

- correct format (4 different options, answer A-D);
- the evidence sentence really exists in the PDF;
- the question is about the evidence sentence;
- the answer letter is set from the option that best matches the evidence (the model often mislabels it);
- exactly one option is true, each option checked on its own against the passage and related chunks;
- not too easy (two or more options with "always/never" are rejected);
- no two questions use the same fact.

Rejected questions are logged (`show_rejects()`), so a short quiz can be explained.

Part 9: Summary tool. Whole PDF: summarize each page, then summarize the summaries (map-reduce). One topic: search, then summarize only the closest chunks, focused on that topic. The result is cached.

Part 10: The agent. A router prompt with examples makes the model return JSON naming a tool. If the output is broken, keyword rules take over. Exactly one tool runs per message, so it cannot loop. For follow-up questions, the last chat messages are used to rewrite "it", "this" or "that" into a full question. Generic topics like "the PDF" are treated as no topic.

Part 11: Web page. Gradio with two tabs. Chat uses the agent. Quiz has a slider (1-20 questions), an optional topic, a progress bar, one question at a time with instant ✅/❌ and the PDF sentence as the reason, Back/Next, a ❌/✅ counter, and a final score with the questions to review.

4. Evaluation results

Measure	Result
Right page in top 4	100%
Correct answers on PDF questions	97% (37 / 38)
Off-topic and trap questions refused	100% (8 / 8)
Overall	98% (45 / 46)
Answers using outside knowledge	0
Time for 46 questions (T4 GPU)	about 50-60 seconds

Model comparison on the same chunks: the 3B model also reached 100% on the keyword test, but 3 of its answers used outside knowledge. The 8B model had none, so the 8B model was kept.

5. Problems found and how they were fixed

Problem	Fix
The "I don't know" check compared the wrong score	The guard now reads the reranker score, and the cutoff is calibrated from the test set
The model answered from its own memory (for example, defining "vector database" although the PDF does not)	Grounding check, then a verbatim quote or "I don't know"
"Summarize the PDF" searched for the word "PDF" and failed	Generic words are treated as "no topic"
Chat memory did not work on the web page	The chat history is now passed to the agent, with a worked example in the rewrite prompt
Quiz questions with a wrong answer key	Answer letter set from the evidence sentence
Quiz questions with two true answers	Each option is checked on its own against related chunks
Quiz explanation cut off mid-sentence	The whole PDF sentence is shown
Silly wrong options and repeated facts	"Always/never" rule, one question per fact
Summaries mixing two topics	Headings start a new chunk, and only chunks close to the best match are used
Reference lists in search results	The references section is cut
A fixed wait after starting Ollama and Pinecone counts that update late	The code waits until each service really answers

6. Limitations

- Written and tuned for one PDF. A second PDF has not been tested yet.
- Keyword scoring is a rough proxy for correctness.
- One test question still fails (a passage with a similar word fools the model).
- Quiz questions can be easy or imprecise, and fewer than requested may be produced.
- Topics the PDF barely mentions get only a short quote.
- The text inside a PDF is trusted. A malicious PDF could contain instructions aimed at the model.
- Some state (chat quiz answers, summary cache) is shared by everyone using the public link.

7. Possible deployment

The public link only works while the Colab notebook runs. For a permanent version:

- Replace Ollama with a hosted LLM API, or rent a GPU, because free hosts have no GPU.
- Add a PDF upload box and a separate Pinecone namespace per visitor.
- Keep API keys in the host's secrets, and add a password or rate limit.
- Check current free-tier and plan rules of the host, because they change.

8. Glossary

- **RAG:** look up relevant text first, then let the model answer from it.
- **Embedding:** a text turned into numbers; similar meaning gives similar numbers.
- **Chunk:** a small piece of the PDF.
- **Vector database:** a store that finds the closest embeddings quickly.
- **Reranker:** a slower, more accurate second ranking of the best candidates.
- **Hallucination:** the model inventing an answer.
- **Grounding:** how much of an answer is really supported by the PDF.
- **Agent / router:** a model that decides which tool to use.
- **Temperature:** randomness of the model (0 = repeatable).